

extract_purpose意图提取插件开发文档

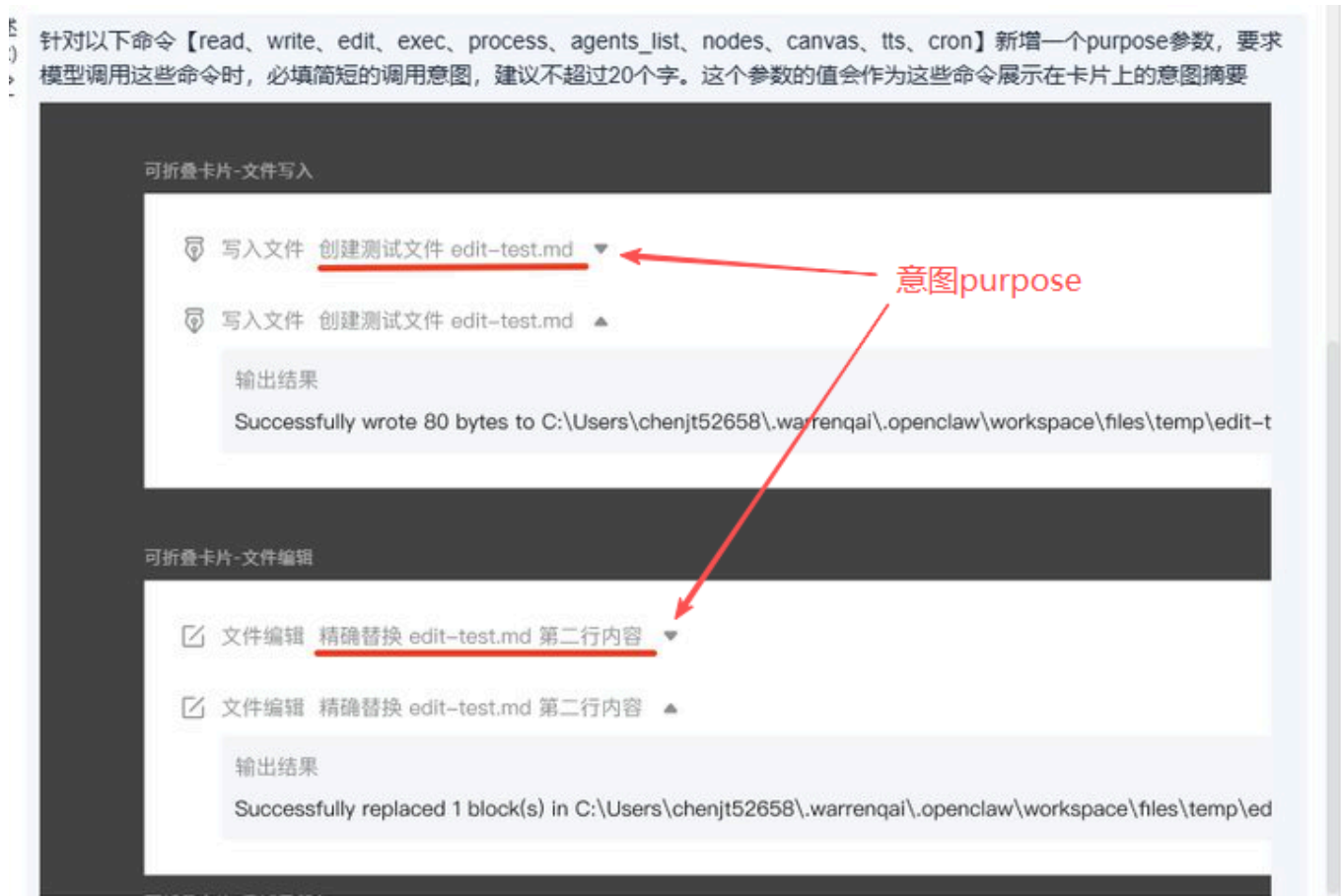
1. 需求背景

在 Agent 思维链（Chain of Thought, CoT）跟踪场景中，需要及时在后台追踪并定位问题。为此，基于 OpenClaw 开发意图提取插件 `extract_purpose`，针对以下工具调用新增 `purpose` 参数：

```
["read", "write", "edit", "exec", "process", "agents_list", "nodes", "canvas", "tts", "cron"]
```

目标：模型调用上述工具时，必须填写简短的调用意图（建议不超过 20 字），该参数值将作为工具调用卡片上的意图摘要进行展示。

目标效果如下：



2. 技术方案

2.1 V1：代理工具方案（已废弃）

方案原理

通过 `api.registerTool()` 注册代理工具（如 `EP-Read-v1`、`EP-Write-v1` 等）替代原生工具。每个代理工具的参数定义中强制包含必填的 `purpose` 字段，LLM 调用代理工具时必须填写意图，插件提取 `purpose` 生成 `task_card`，同时内部执行原始操作返回真正内容。

代理工具映射

代理工具通过声明式数组 `TOOL_MAPPINGS` 统一描述，每个条目定义了原始工具与代理工具的映射关系：

```
TOOL_MAPPINGS = [

    { originalName: 'read',          proxyName: 'EP-Read-v1',          label: '读取  
(含意图)',    paramFields: { file_path } },

    { originalName: 'write',         proxyName: 'EP-Write-v1',        label: '写入  
(含意图)',    paramFields: { file_path, content } },

    { originalName: 'edit',          proxyName: 'EP-Edit-v1',         label: '编辑  
(含意图)',    paramFields: { file_path, old_string, new_string } },

    { originalName: 'exec',          proxyName: 'EP-Exec-v1',         label: '执行命  
令(含意图)',  paramFields: { command } },

    { originalName: 'process',       proxyName: 'EP-Process-v1',      label: '进程操  
作(含意图)',  paramFields: { action } },

    { originalName: 'agents_list',   proxyName: 'EP-AgentsList-v1',  label: '代理列  
表(含意图)',  paramFields: {} },

    { originalName: 'nodes',         proxyName: 'EP-Nodes-v1',        label: '节点操  
作(含意图)',  paramFields: { action } },

    { originalName: 'canvas',        proxyName: 'EP-Canvas-v1',       label: '画布操  
作(含意图)',  paramFields: { action } },

    { originalName: 'tts',           proxyName: 'EP-TTS-v1',          label: '语音合  
成(含意图)',  paramFields: { text } },

    { originalName: 'cron',          proxyName: 'EP-Cron-v1',         label: '定时任  
务(含意图)',  paramFields: { action } },

]
```

每个映射条目包含以下字段：

字段	含义
originalName	原生工具名（如 read）
proxyName	代理工具名（如 EP-Read-v1）
label	中文标签（如"读取（含意图）"）
description	工具描述，提示模型必须填写 purpose
paramFields	原生工具的参数定义（JSON Schema 格式）
executeOriginal	实际执行原始操作的异步函数

工作流程

以 read 工具为例，完整流程如下：

- LLM 发起代理工具调用 EP-Read-v1(purpose="读取配置", file_path="src/index.ts")
- 插件校验 purpose 参数，为空则拒绝并返回错误提示
- 截断 purpose 至 maxPurposeLength 字符
- 从参数中剔除 purpose，收集原始工具参数
- 调用 mapping.executeOriginal() 执行原始操作（如读取文件内容）
- 返回原始内容 + channelData (task_card) + details

返回结构：

```
{

  content: originalResult.content,           // 原始工具的执行结果

  channelData: {

    extractPurpose: {

      type: 'task_card',                     // 前端渲染类型

      purpose: truncatedPurpose,             // 意图摘要

      originalTool: mapping.originalName,    // 原始工具名

      proxyTool: mapping.proxyName,         // 代理工具名

      originalParams,
```

```
    status: 'completed', // 状态

    timestamp: Date.now(), // 时间戳

  },

  details: { source, purpose, originalTool, originalParams, ... }

}
```

注：V1 的目标工具列表为硬编码常量 `TARGET_TOOL_NAMES`，不支持通过配置项自定义。

2.2 V2: Hook 拦截方案（当前采用）

方案原理

V1 方案存在固有缺陷：LLM 可能直接调用原生工具名（如 `read`）而跳过代理工具（如 `EP-Read-V1`），导致意图提取被绕过。V2 方案通过 OpenClaw 的 `before_tool_call` / `after_tool_call` 钩子机制拦截所有工具调用，无论 LLM 调用哪个工具，钩子都会触发，从而彻底解决绕过问题。

工作流程

以 `read` 工具为例，完整流程如下：

1. LLM 发起工具调用 `read(file_path="src/index.ts")`
2. `before_tool_call` 钩子触发，从参数中提取意图，生成 `purpose`（如“读 `src/index.ts`”）
3. 将 `purpose` 以 `runId` 为键存入 `pendingPurposes Map`
4. 原生工具 `read` 正常执行，返回结果
5. `after_tool_call` 钩子触发，从 `Map` 中取出 `purpose`，注入到结果的 `channelData` 中
6. 前端读取 `channelData.purpose` 渲染意图徽章

意图提取算法

采用两层逻辑，优先保证语义完整性：

第 1 层：模型显式声明意图

若模型在调用参数中提供了 `params.purpose` 字段，则直接沿用，不做任何压缩，保证语义完整。

第 2 层：从参数推导意图

若未提供 `purpose` 字段，则综合工具参数按优先级推导，拼接为“动作 + 目标”格式：

- 1. **路径类参数**（file_path、path、url 等）→ 提取人类可读的短路径（优先展示文件名，必要时追加父目录，如 src/index.ts）
- 2. **命令类参数**（command、cmd、script）→ 提取命令主体及首个参数
- 3. **内容类参数**（content、text、query、old_string、new_string 等）→ 截取前 7 字加省略号
- 4. **动作类参数**（action、type、method）→ 直接取值
- 5. **Cron 表达式**（cron）→ 直接取值

推导结果若超出最大长度限制，则通过 compressPurpose 进行压缩：

- 1. 去除中文冗余词（"进行"、"操作"、"的了"等）
- 2. 若包含多个逗号/分号分隔的动作，仅保留第一个核心动作
- 3. 仍超长则保留首尾关键信息，中间以 ... 连接

配置项说明

插件在 openclaw.plugin.json 中定义了以下可配置项：

配置项	类型	默认值	说明
maxPurposeLength	number	20	意图摘要的最大字符长度，最小值为 5
targetTools	string[]	["read", "write", "edit", "exec", "process", "agents_list", "nodes", "canvas", "tts", "cron"]	需要拦截并提取意图的工具名称列表

3. 项目结构

```
extract_purpose/

├─ index.ts                # 核心源码（插件全部逻辑）

├─ openclaw.plugin.json    # 插件元数据与配置项定义

├─ package.json            # npm 包信息、构建脚本、依赖声明

├─ tsconfig.json           # TypeScript 编译选项

└─ dist/

    └─ index.js            # 编译产物 - 运行时代码（OpenClaw 实际加载）
```

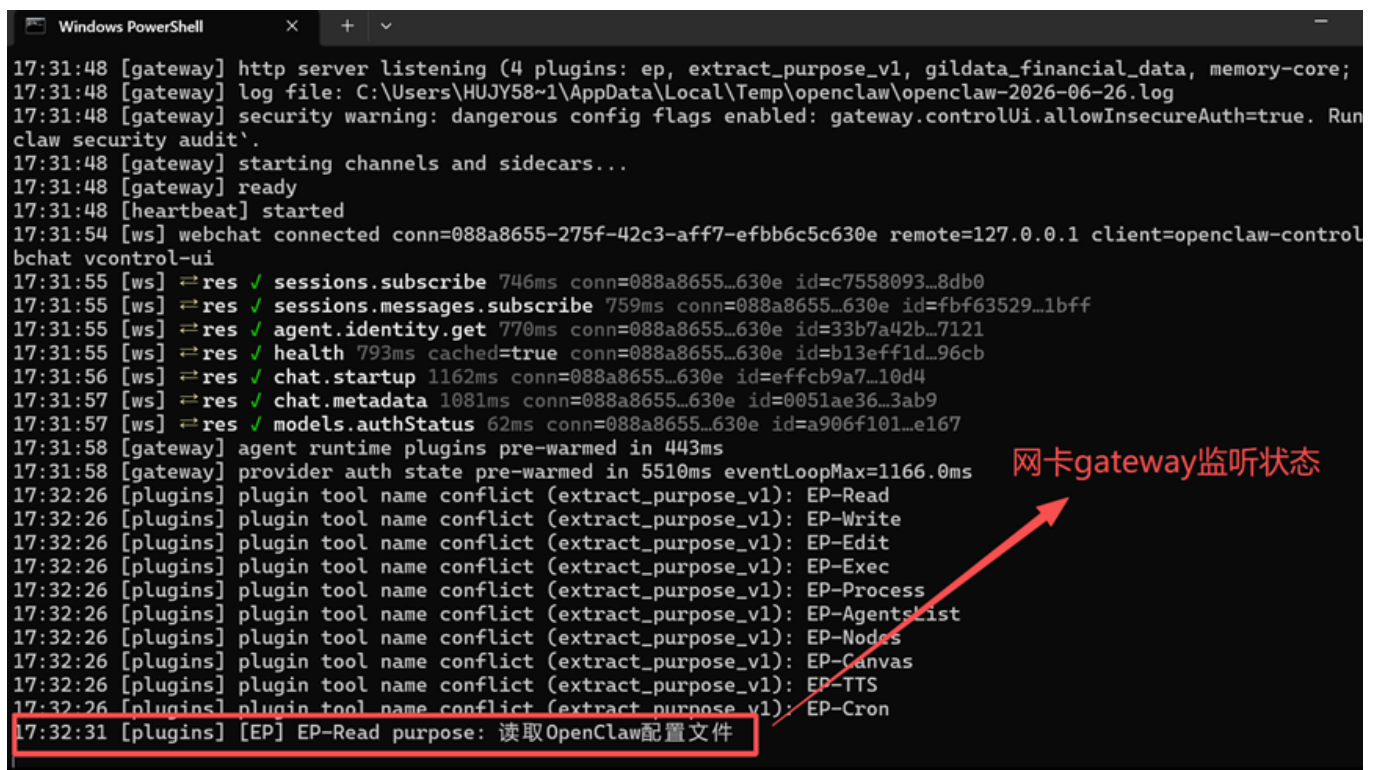
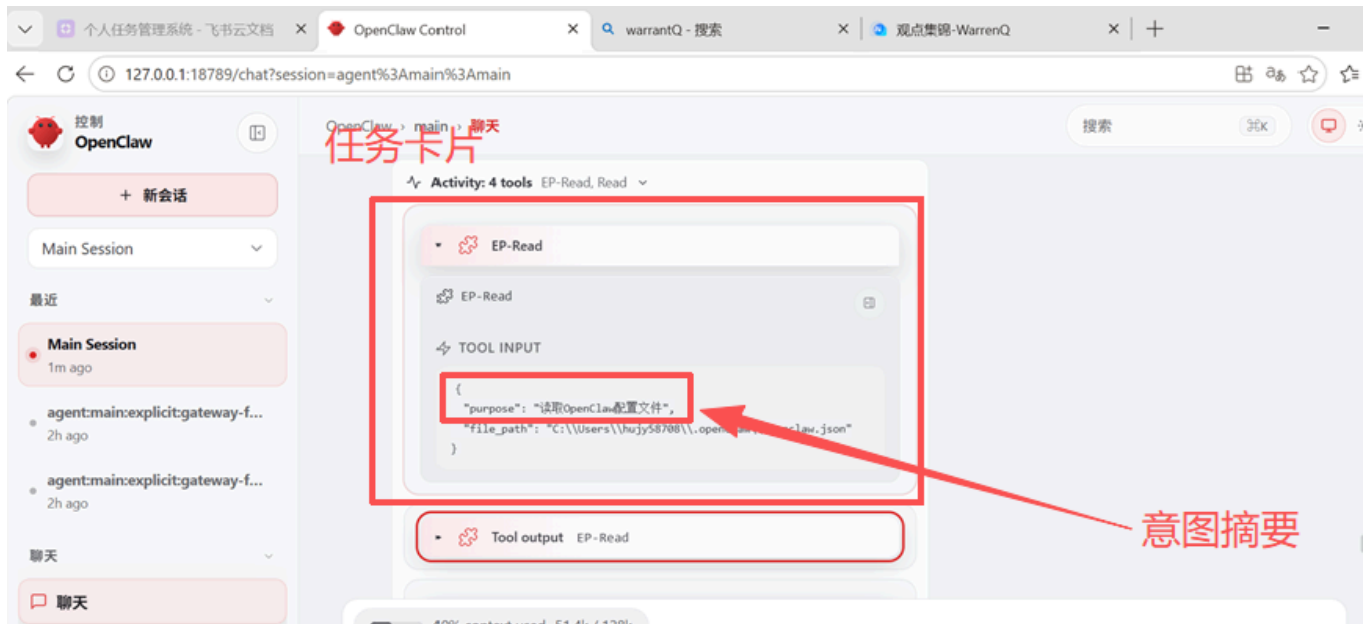
└─ index.d.ts

编译产物 - 类型声明（供其他 TS 项目引用）

4. 运行效果

4.1 V1 效果

在客户端中要求读取文件时，会先调用代理工具 EP-Read-V1 提取意图并在前端渲染任务卡片，网关 Gateway 监听信息得到工具 EP-Read-V1 的信息，可以看到同步构建了 purpose 参数。



4.2 V2 效果

通过 Hook 机制确保每次工具调用均被拦截并进行意图提取，生成 purpose 参数，彻底解决了 V1 方案中代理工具可能被绕过的问题。

```
17:26:57 [plugins] [ExtractPurpose] Plugin activated. targetTools=[read,write,edit,exec,process,agents_list,nodes,canvas,
, tts,cron] maxPurposeLength=20
17:26:57 [gateway] agent runtime plugins pre-warmed in 230ms
17:27:36 [plugins] [ExtractPurpose] DEBUG before_tool_call: tool=exec params={"command":"Add-Content -Path \"C:\\Users\\
huji58708\\Desktop\\测试文件.txt\" -Value \"`r`nhello world\"","purpose":"向桌面测试文件末尾追加一行 hello world"} runId
=a9405512-886e-4382-9399-fa2c783eb552
17:27:36 [plugins] [ExtractPurpose] before_tool_call: tool=exec purpose="向桌面测试文件末尾追加一行 hello world" runId=a
9405512-886e-4382-9399-fa2c783eb552
17:27:37 [plugins] [ExtractPurpose] after_tool_call: injected purpose="向桌面测试文件末尾追加一行 hello world" into chan
nelData for tool=exec
17:29:25 [plugins] [ExtractPurpose] DEBUG before_tool_call: tool=exec params={"command":"$timestamp = Get-Date -Format \"
yyyyMMddHHmm\"; $filename = \"新建文件_${timestamp}.txt\"; New-Item -Path \"C:\\Users\\huji58708\\Desktop\\$filename\"
-ItemType File","purpose":"在桌面创建一个以当前日期时间命名的新文本文件"} runId=f1eda877-21be-4066-b098-6996f6588d38
17:29:25 [plugins] [ExtractPurpose] before_tool_call: tool=exec purpose="在桌面创建一个以当前日期时间命名的新文本文件" r
unId=f1eda877-21be-4066-b098-6996f6588d38
17:29:26 [plugins] [ExtractPurpose] after_tool_call: injected purpose="在桌面创建一个以当前日期时间命名的新文本文件" int
o channelData for tool=exec
```

purpose意图提取

hook拦截

5. 版本演进说明

版本	方案	状态	说明
V1	代理工具（EP-Read-V1 等）	已废弃	LLM 可能直接调用原生工具名而跳过代理工具，导致意图提取被绕过
V2	Hook 拦截（before_tool_call / after_tool_call）	当前采用	钩子在工具调用层面拦截，无论 LLM 调用哪个工具均会触发，彻底解决绕过问题